

Rapport technique — API de gestion d'événements culturels

Projet d'évaluation finale — Master 1 Intelligence Artificielle & Big Data

Auteur : Folly S. Carlo GBOSSOU

Email : carlogbossou93@gmail.com

Date : Juin 2026

Dépôt : GitHub (branche master)

Démo : <https://events-api.gfolly.com/docs>

1. Présentation du projet

Ce projet consiste en une API REST publique permettant de gérer des événements culturels — concerts, pièces de théâtre et conférences. L'API est destinée à des développeurs tiers et expose une validation rigoureuse des données, une authentification basée sur JWT et un audit trail automatique de toutes les opérations d'écriture.

1.1 Périmètre fonctionnel

L'API permet de :

- **Gérer des organisateurs** : création, consultation, modification et suppression d'entités organisatrices.
- **Gérer des salles** : lieux physiques avec adresse, capacité et coordonnées géographiques.
- **Gérer des catégories thématiques** : jazz, théâtre contemporain, conférence tech, etc.
- **Gérer des événements polymorphes** : un événement est soit un Concert (avec `artist`, `genre`), soit un Theatre (avec `director`, `cast_members`), soit une Conference (avec `speaker`, `topic`).
- **Rechercher et filtrer** les événements par ville, type, catégorie, organisateur, plage de dates, prix maximum et tags.
- **S'authentifier** via JWT avec trois niveaux de droits : USER, ORGANIZER, ADMIN.

L'API expose 24 endpoints documentés dans Swagger UI.

2. Choix d'architecture

2.1 DDD Lite

L'architecture retenue est un **Domain-Driven Design allégé (DDD Lite)**, organisé en modules métier indépendants. Chaque module (`iam`, `events`, `organizers`, `venues`, `categories`) est découpé en quatre couches :

module/

```

└─ domain/      # Entités, règles métier, interfaces repositories
└─ application/ # UseCases, mappers (orchestration sans dépendance
technique)
└─ api/         # Contrôleurs FastAPI, DTOs (Create, Update, Read,
ReadDetail)
└─ infrastructure/ # Implémentation SQLAlchemy des repositories

```

La règle de dépendances est stricte :

```

api → application → domain
      ↑
    infrastructure

```

La couche `domain` ne dépend d'aucune autre couche. Cette séparation garantit que la logique métier reste testable indépendamment du framework.

2.2 Pourquoi DDD Lite plutôt que Clean Architecture complète ?

La Clean Architecture impose des couches supplémentaires (interfaces d'entrée/sortie explicites, value objects distincts des entités) qui alourdisent considérablement la structure pour un projet de cette taille. Le DDD Lite conserve les bénéfices essentiels — testabilité, séparation des responsabilités, indépendance du domaine — sans la verbosité d'une architecture de type hexagonale complète.

2.3 Stack technique

Composant	Choix	Justification
Framework	FastAPI 0.115+	Typage natif, OpenAPI automatique, performances async
Validation	Pydantic v2	Types personnalisés via <code>__get_pydantic_core_schema__</code> , v2 est 10x plus rapide que v1
ORM	SQLAlchemy 2 (async)	Support natif asyncio, syntaxe déclarative moderne
Base de données	PostgreSQL 16	ACID, JSONB, robustesse en production
Migrations	Alembic	Intégration native SQLAlchemy
Driver async	asyncpg	Driver natif asyncio le plus performant pour PostgreSQL
Auth	JWT (python-jose) + bcrypt	Standard de l'industrie, stateless
Tests	pytest + pytest-asyncio + Hypothesis	Tests unitaires, d'intégration et property-based
Qualité	Ruff + Black + MyPy strict	Lint, formatage et typage statique en un pipeline
Déploiement	Docker + Coolify	Conteneurisation reproductible, PaaS auto-hébergé

2.4 Asynchronisme

L'ensemble de la pile I/O est asynchrone : FastAPI, SQLAlchemy 2 avec `AsyncSession`, `asyncpg`. Chaque requête HTTP ne bloque pas de thread système, ce qui améliore la concurrence sous charge.

3. Fonctionnalités avancées

3.1 Types Pydantic v2 personnalisés

Cinq types métier ont été implémentés en sous-classant `str` et en fournissant un `__get_pydantic_core_schema__` conforme à l'API Pydantic v2 :

Type	Validation appliquée
Slug	Regex <code>^[a-z0-9]+(?:-[a-z0-9]+)*\$</code> — identifiant URL-safe
URLImage	HTTPS obligatoire, extensions autorisées : <code>jpg</code> , <code>jpeg</code> , <code>png</code> , <code>webp</code>
CodePostalFR	Exactement 5 chiffres, département 01–95 (hors DOM-TOM)
TelephoneE164	Format international +<indicatif><numéro>, longueur totale 8–15 chiffres
IBAN	Validation de la longueur par pays + algorithme MOD-97 sur le checksum

Chaque type expose une méthode de classe `_validate` qui est également le point d'entrée des tests Hypothesis.

Le type `Slug` fournit en outre une méthode utilitaire `Slug.from_title(title)` qui génère automatiquement un slug valide depuis un titre quelconque en normalisant les accents (NFD) et en substituant les caractères non-URL.

3.2 Polymorphisme des événements

Le polymorphisme est implémenté via le **discriminator field** de Pydantic v2. Le champ `event_type` (valeur littérale `"concert"`, `"theatre"` ou `"conference"`) permet à Pydantic de sélectionner le bon schéma de validation au moment de la désérialisation :

```
EventCreate = Annotated[
    ConcertCreate | TheatreCreate | ConferenceCreate,
    Field(discriminator="event_type"),
]
```

FastAPI expose ce type union directement dans OpenAPI via `Body(discriminator="event_type")`, ce qui génère un Swagger UI avec trois exemples distincts selon le type d'événement. Le même pattern est appliqué aux schémas de réponse

(EventRead, EventReadDetail).

En base de données, la table `events` est unique (table par hiérarchie, stratégie STI) avec des colonnes spécifiques nullable pour chaque sous-type.

3.3 Filtres avancés et pagination

L'endpoint `GET /api/v1/events` supporte huit paramètres de filtre cumulables :

```
GET /api/v1/events
?city=Paris
&event_type=concert
&category_public_id=<uuid>
&organizer_public_id=<uuid>
&date_min=2025-07-01T00:00:00Z
&date_max=2025-12-31T23:59:59Z
&price_max=50
&tags=jazz,bebop
&sort_by=start_at
&sort_order=asc
&page=1
&size=20
```

Le filtre sur les tags utilise un `ILIKE` cumulatif — chaque tag doit être présent dans la colonne `tags` — ce qui évite d'ajouter une table de jointure pour un cas d'usage simple.

La pagination est implémentée côté base de données (SQL `LIMIT / OFFSET`) avec retour du `total` dans la réponse pour permettre le calcul du nombre de pages côté client.

3.4 Audit trail et logs structurés

Toute opération d'écriture (création, modification, suppression) est tracée de deux façons :

1. **Persistence en base** : chaque événement est enregistré dans la table `audit_logs` avec `entity_type`, `entity_public_id`, `action`, `actor_public_id`, `timestamp` et un champ `details` JSON contenant l'état avant/après.
2. **Log JSON sur stdout** : un `JsonFormatter` personnalisé sérialise chaque enregistrement de log Python en JSON avec `timestamp` (ISO 8601 UTC), `level`, `logger`, `message`, `module`, `function`, `line` et les champs extra de l'audit.

```
{
  "timestamp": "2025-07-14T18:30:00.123456+00:00",
  "level": "INFO",
  "logger": "audit",
  "message": "audit",
  "entity_type": "event",
  "entity_public_id": "550e8400-e29b-41d4-a716-446655440000",
  "action": "CREATE",
  "actor": "a3f2c1d0-1234-5678-abcd-ef0123456789"
}
```

Ce format est directement ingérable par des outils comme Loki, Elasticsearch ou Datadog sans pré-traitement.

3.5 Sécurité

- **Aucun secret en clair dans le dépôt** : `.env` est dans `.gitignore`, `.env.example` fournit la structure sans valeurs sensibles.
 - **Hachage des mots de passe** : `bcrypt` via `passlib`.
 - **Validation stricte des entrées** : `Pydantic v2` rejette tout champ non déclaré (`model_config = {"extra": "forbid"}`).
 - **RBAC** : les dépendances FastAPI `require_admin`, `require_organizer`, `get_current_user` vérifient le rôle extrait du JWT à chaque requête.
 - **Soft delete** : les entités supprimées ne sont pas physiquement effacées mais marquées `deleted=True`, préservant l'historique pour l'audit.
-

4. Organisation du code et métriques

4.1 Structure

Le projet compte **15 commits** avec un historique conventionnel (`feat:`, `chore:`, `docs:`) organisé en 12 phases de développement incrémentales.

```
eventmanagement-project/
├── app/
│   ├── modules/           # 5 modules métier (iam, events, organizers, venues,
│   │   │   │               categories)
│   ├── shared/            # Types, pagination, exceptions communes, BaseEntity
│   └── infrastructure/    # Config, BDD, JWT, logs, audit
├── tests/
│   ├── unit/              # 6 fichiers – types personnalisés + UseCases
│   └── integration/       # 4 fichiers – endpoints HTTP (auth, venues,
│   │   │   │               categories, health)
│   └── property/          # 1 fichier – 10 tests Hypothesis sur les 5 types
├── docs/                  # Architecture, règles, cahier des charges, Postman
├── Dockerfile
├── docker-compose.yml     # Production (PostgreSQL externe Coolify)
├── docker-compose.local.yml # Local autonome (PostgreSQL intégré)
└── .github/workflows/ci.yml # Pipeline CI : lint + tests + build Docker
```

4.2 Métriques clés

Indicateur	Valeur
Endpoints exposés	24
Modules métier	5
Types personnalisés Pydantic v2	5
Types d'événements polymorphes	3
Fichiers de tests	11
Tests Hypothesis (property-based)	10
Seuil de couverture configuré	80 %

Indicateur	Valeur
Commits	15
Python requis	≥ 3.13

4.3 Pipeline qualité

Le pipeline CI GitHub Actions exécute trois jobs en série :

1. **Lint** : `ruff check app/` (200+ règles) + `black --check app/`
 2. **Test** : démarrage d'un service PostgreSQL 16, exécution des migrations Alembic, `pytest` avec rapport de couverture XML, upload vers Codecov
 3. **Docker** : build de l'image pour valider le `Dockerfile`
-

5. Difficultés rencontrées

5.1 API Pydantic v2 pour les types personnalisés

La migration de Pydantic v1 vers v2 a profondément changé la façon de définir des types personnalisés. La méthode `__get_validators__` de v1 n'existe plus ; il faut implémenter `__get_pydantic_core_schema__` qui retourne un objet `CoreSchema` de `pydantic_core`.

L'intégration du serializer (pour que le type soit sérialisé en `str` simple dans les réponses JSON) nécessite un `plain_serializer_function_ser_schema`, ce qui n'est pas documenté de façon immédiate. Cette partie a demandé une lecture approfondie du code source de Pydantic v2.

5.2 Discriminator avec FastAPI et OpenAPI

Le discriminator Pydantic fonctionne bien pour la validation, mais son intégration dans OpenAPI via FastAPI nécessite d'utiliser `Body(discriminator="event_type")` explicitement dans la signature du contrôleur — le seul typage `Annotated[... , Field(discriminator=...)]` ne suffit pas à générer le bon schéma OpenAPI. Cette subtilité a été découverte lors de la vérification du Swagger UI.

5.3 Tests asyncio avec pytest-asyncio

La configuration `asyncio_mode = "auto"` et `asyncio_default_fixture_loop_scope = "session"` de `pytest-asyncio` ≥ 0.24 diffère des versions antérieures. La session de test partage une seule boucle asyncio, ce qui impose d'utiliser `NullPool` pour SQLAlchemy afin d'éviter les conflits de connexion entre fixtures de portée différente (`session` vs `function`).

5.4 Génération d'IBAN valides pour Hypothesis

L'algorithme MOD-97 de validation des IBAN imposait de générer des valeurs avec un checksum

exact. Hypothesis ne peut pas le faire directement avec `st.from_regex`. La solution a été d'écrire une stratégie composée : générer le BBAN aléatoire, calculer le checksum correct, puis construire l'IBAN complet — en s'assurant que les caractères alphabétiques sont correctement convertis en valeurs numériques pour le calcul.

5.5 Soft delete et filtres

S'assurer que toutes les requêtes SQLAlchemy filtrent systématiquement sur `deleted.is_(False)` (et non `== False`, qui ne gère pas correctement le `NULL`) a requis une attention particulière lors de chaque nouveau repository. Le choix d'utiliser `.is_(False)` plutôt que `== False` est une exigence SQLAlchemy pour les colonnes booléennes nullables.

6. Déploiement

6.1 Docker

L'application est conteneurisée avec un `Dockerfile` Python 3.13-slim. L'image installe les dépendances depuis `pyproject.toml`, expose le port 8000 et exécute automatiquement les migrations Alembic au démarrage avant de lancer Uvicorn.

Deux configurations Docker Compose sont fournies :

- `docker-compose.local.yml` : autonome avec un service PostgreSQL 16 Alpine intégré, health check et `depends_on`. Commande : `docker compose -f docker-compose.local.yml up --build`.
- `docker-compose.yml` : API seule, `DATABASE_URL` injectée par la plateforme externe (Coolify).

6.2 Coolify (production)

L'API est déployée en production sur une instance Coolify auto-hébergée. Coolify gère automatiquement le build de l'image Docker depuis le dépôt Git, l'injection des secrets (variables d'environnement) et le reverse proxy HTTPS. La base de données PostgreSQL est également gérée par Coolify dans un conteneur dédié.

L'API est accessible publiquement à <https://events-api.gfolly.com>.

7. Perspectives et améliorations envisageables

7.1 Court terme

- **Compléter les tests d'intégration** pour les modules `events` et `organizers` afin d'assurer une couverture uniforme sur l'ensemble des endpoints.
- **Rate limiting** : ajouter `slowapi` pour protéger les endpoints publics contre les abus.
- **Refresh tokens** : le système JWT actuel n'implémente pas de refresh token ; les sessions

expirent définitivement après `ACCESS_TOKEN_EXPIRE_MINUTES`.

7.2 Moyen terme

- **Images réelles** : stocker les images événement sur un object storage (S3 ou compatible) plutôt qu'une simple URL. Le type `URLImage` serait étendu pour valider le domaine autorisé.
- **Recherche full-text** : intégrer la recherche PostgreSQL `tsvector/tsquery` sur les champs `title`, `description` et `tags` pour des résultats plus pertinents que le `ILIKE`.
- **Notifications** : système d'alertes (email, webhook) lorsqu'un événement passe à l'état `published` ou `sold_out`.

7.3 Long terme

- **Microservices** : à forte échelle, les modules `events` et `iam` pourraient être extraits en services indépendants communiquant via un bus de messages (RabbitMQ, Kafka).
- **Cache** : mise en cache Redis des listes d'événements très consultées avec invalidation sur écriture.
- **Internationalisation** : les messages d'erreur de validation sont actuellement en français ; une couche `i18n` permettrait de cibler des développeurs tiers non francophones.

8. Conclusion

Ce projet a permis de mettre en pratique l'ensemble des bonnes pratiques du développement d'API professionnelles avec FastAPI et Pydantic v2 : architecture en couches, typage strict, tests automatisés à plusieurs niveaux (unitaire, intégration, property-based), documentation OpenAPI riche et déploiement reproductible.

Les fonctionnalités exigées par le cahier des charges — types personnalisés, polymorphisme par discriminator, filtres avancés, audit trail, pipeline CI — ont toutes été implémentées. Le projet est déployé en production et accessible publiquement.

Les principales difficultés ont porté sur la maîtrise de la nouvelle API de Pydantic v2 pour les types personnalisés et sur la gestion du contexte `asyncio` dans les tests. Ces obstacles ont été surmontés par une lecture approfondie de la documentation officielle et des tests itératifs.